

YOLOv7 Optimization and Profiling Report

AMD/Xilinx Versal VEK280 (VE2802 NPU)

Subject: End-to-end latency, NPU/AIE activity and power profiling

Platform: VEK280, VE2802, 304 AIE tiles, 500 MHz, AIEML_V1C backend

Model: YOLOv7, fixed 640 x 640 input

Date: October 2026

Executive Summary

This report documents the optimization and stage-level profiling of the YOLOv7 object-detection pipeline on the AMD/Xilinx Versal VEK280 board. The goals were to separate NPU time from CPU-side processing, quantify how source-image resolution affects end-to-end latency, measure AIE activity and power, and identify the dominant bottleneck.

Baseline (VART)	NPU-only runner	NPU stage (profiled)	Best E2E throughput
175.4 ms 5.7 FPS	73.75 ms 13.56 FPS	~7.1 ms constant	27.01 FPS (320x240 source)

Key findings

- **The NPU is not the bottleneck.** NPU inference takes ~7.1 ms per 640 x 640 frame regardless of whether the source image is 320x240 or 1920x1080.
- **Moving off the CPU subgraph cut latency by 2.4x** (175.4 ms to 73.75 ms) in the VART runner, and the custom pipeline reaches 13.3 to 27.0 FPS end-to-end depending on source resolution.
- **Image read/decode dominates at high resolution:** 44.31 ms at 1920x1080, about 59% of frame latency and 6.2x the NPU stage.
- **Input preparation is a fixed ~12.2 to 12.8 ms cost**, larger than the NPU stage itself, at every resolution.
- **CPU load rises from 72% to 95%** as resolution grows, confirming a CPU-bound input pipeline.
- **AIE core activity averaged 52.48%** across all 304 tiles (range 52.21% to 52.77%). AIE profiling adds large overhead and must not be used for benchmarking.

Recommendation: concentrate further optimization on JPEG read/decode, resize and input preparation (and on overlapping these stages with NPU execution). Further NPU-level tuning will yield little end-to-end gain.

1. Objective and Scope

The work targeted improved YOLOv7 execution on the VEK280 NPU, separation of NPU time from CPU-side time, profiling of each pipeline stage, evaluation of the effect of source-image resolution, measurement of AIE/NPU activity, examination of power, and identification of the main limiter of end-to-end FPS. The YOLOv7 network always receives a fixed 640 x 640 input irrespective of the original image size.

2. Test Platform and Methodology

Item	Detail
Board / device	AMD/Xilinx Versal VEK280, VE2802 NPU configuration
AIE array	304 tiles (38 x 8), 500 MHz, backend AIEML_V1C, kernel npu_aieml_OO
Model input	Application: (1,3,640,640) float32 NCHW. Native NPU: (1,640,640,4) INT8 NHWC4
Model outputs	[1,80,80,256], [1,40,40,256], [1,20,20,256] (NHWC8)
Profiling app	Custom stage-level profiler; 101 images per resolution
Resolutions tested	320x240, 640x480, 1280x720, 1920x1080 (JPEG sources)
Stages timed	Read/decode, letterbox, preparation, NPU, output decode, NMS, draw, frame total
Other probes	XRT AIE profiling (core active / group stall counters); INA226 rail power monitoring

The pipeline under test is:

Read/decode > Letterbox > Input preparation > NPU inference > Output decode > NMS > Draw

3. Optimization Progression

Baseline. The standard VART path executed a CPU-side ONNX subgraph (wrp_network_embd_export_2_cpu_subgraph_call), yielding 175.4 ms/frame (~5.7 FPS). This figure is not representative of NPU-only execution.

NPU-only mode. Running vart_ml_runner.py with --npu_only skips the CPU subgraph. Average latency fell to ~73.75 ms (~13.56 FPS), a 2.38x reduction, showing the CPU subgraph was a major contributor. This number still includes runner and input-handling overhead and is not pure NPU hardware latency; the profiled NPU stage is ~7.1 ms.

Native input path. The profiling path feeds the NPU its native INT8 NHWC4 tensor, avoiding extra conversion work before execution and enabling the stage-level breakdown below.

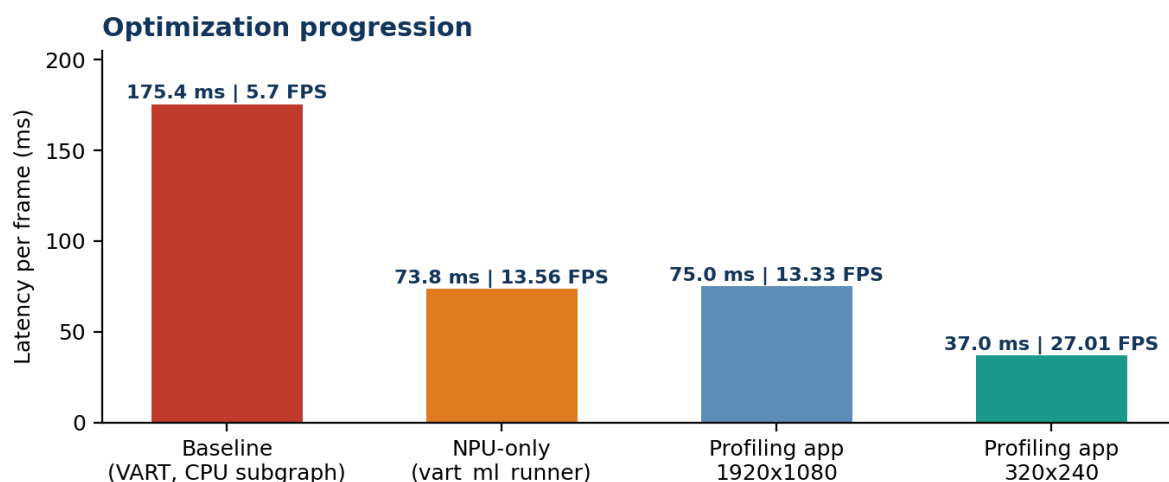


Figure 1. Latency progression. The first two bars come from the VART runner; the last two from the custom profiling application and so are not strictly like-for-like.

4. Stage-Level Profiling Results

All values are averages over 101 images per resolution. The NPU model input is 640 x 640 in every case.

Stage (ms)	320x240	640x480	1280x720	1920x1080
Read/decode	9.52	16.07	30.50	44.31
Letterbox	4.58	0.69	3.17	7.97
Preparation	12.78	12.41	12.25	12.22
NPU	7.18	7.14	7.12	7.12
Output decode	2.39	2.55	2.44	2.42
NMS	0.24	0.25	0.24	0.24
Draw	0.31	0.44	0.57	0.69
Frame total	37.02	39.57	56.32	74.99
End-to-end FPS	27.01	25.27	17.76	13.33
CPU busy	72%	72%	80%	95%

Table 1. Per-stage average latency by source resolution. Stage sums match frame totals to within ~ 0.03 ms.

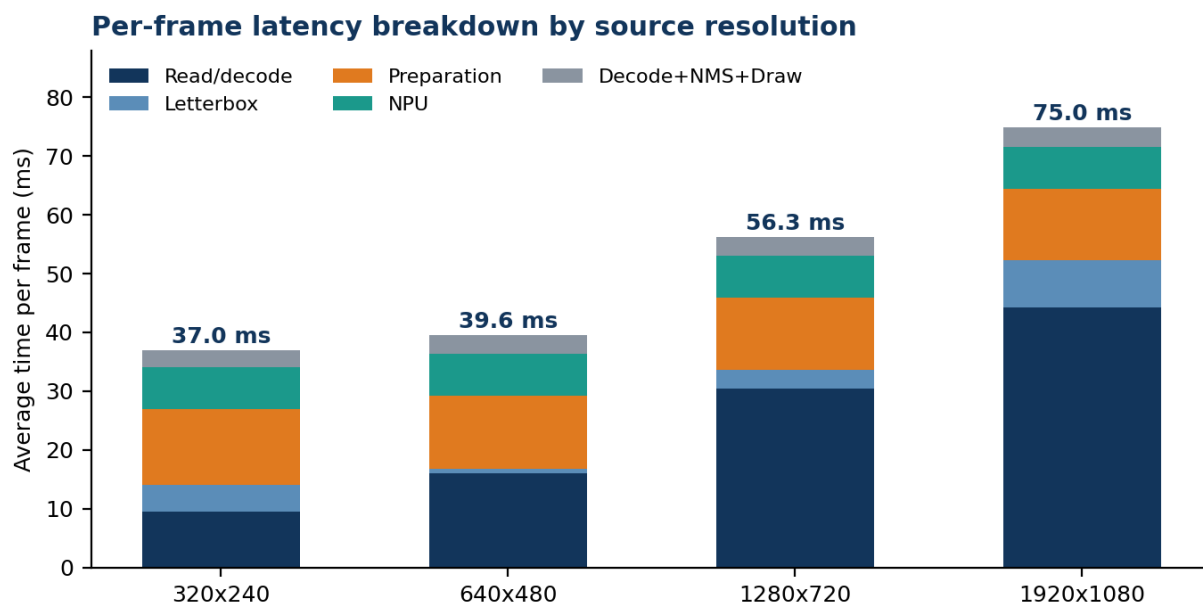


Figure 2. Stacked per-frame breakdown. The NPU segment (teal) is constant while read/decode grows.

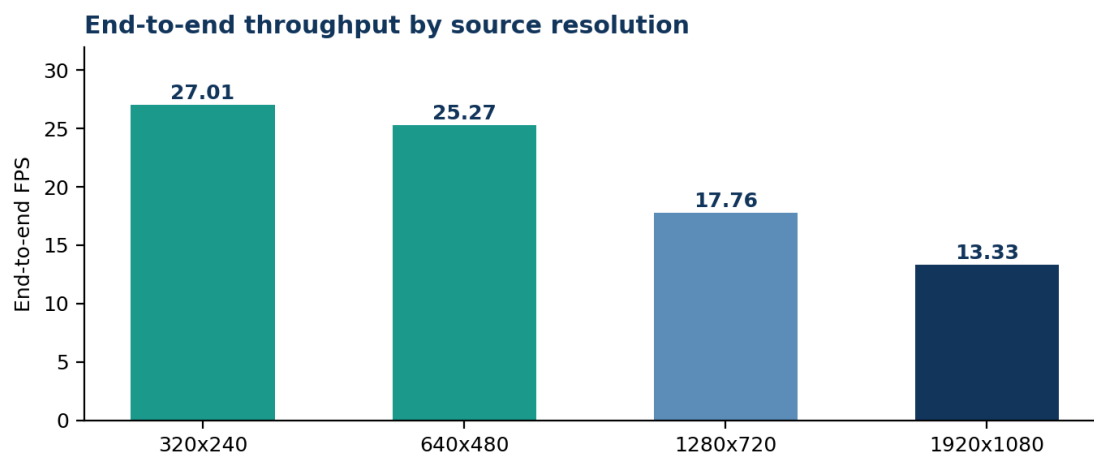


Figure 3. End-to-end FPS falls from 27.0 to 13.3 as source resolution increases.

5. Analysis

5.1 NPU latency is resolution-independent

NPU time stays within 7.12 to 7.18 ms across a 12x change in source pixel count. Because letterboxing always produces a 640 x 640 tensor, the neural-network workload is identical. The theoretical NPU-only ceiling implied by 7.12 ms is ~140 inferences per second, roughly 10x the best measured end-to-end rate, so the NPU is heavily under-fed by the CPU pipeline.

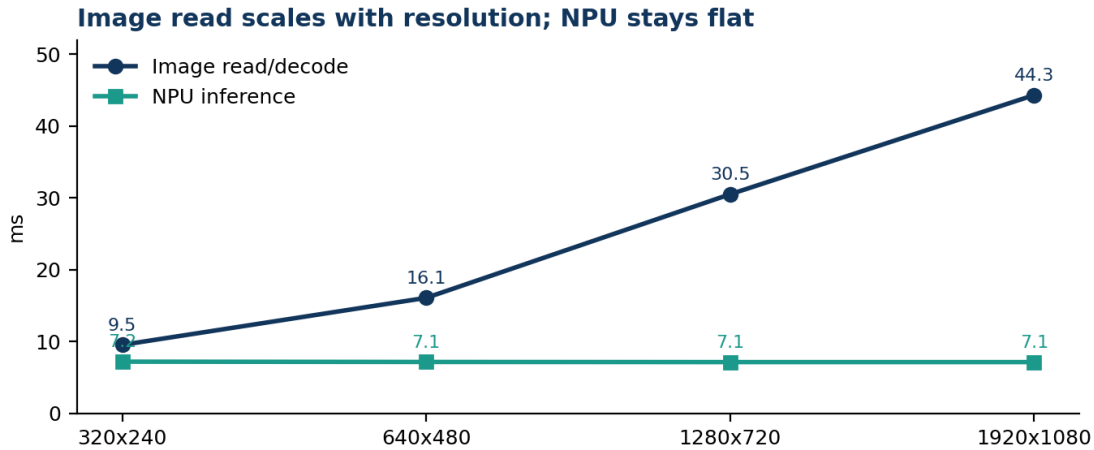


Figure 4. Read/decode versus NPU time.

5.2 Image read/decode drives the resolution penalty

Read time rises 4.7x (9.52 ms to 44.31 ms) from the smallest to largest source. At 1920x1080 it accounts for ~59% of frame latency (44.31 / 74.99 ms), and letterbox adds another 7.97 ms (~11%). Together, read and letterbox are about 70% of the frame at this resolution.

5.3 Input preparation is a hidden fixed cost

Preparation takes ~12.2 to 12.8 ms at every resolution, which is 1.7x the NPU stage. Since it does not scale with source size, it presumably reflects work done on the fixed 640 x 640 tensor (e.g. conversion to the INT8 NHWC4 layout). Even at 320x240, read + letterbox + preparation total ~26.9 ms against 7.18 ms of NPU time, so the NPU accounts for only ~19% of the frame. This stage is a strong candidate for optimization at all resolutions.

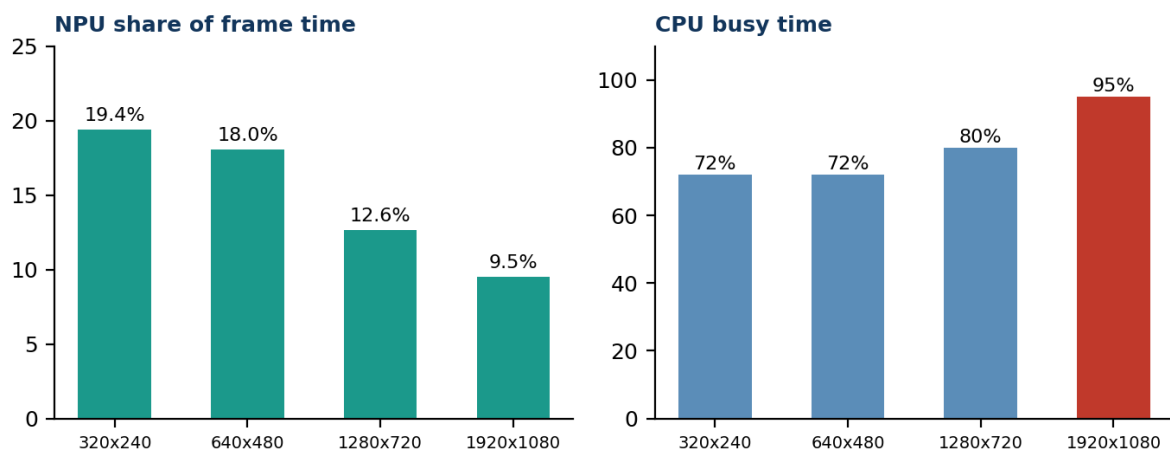


Figure 5. NPU share of frame time (left) and CPU busy time (right).

5.4 Post-processing is negligible

Output decode (~2.4 ms), NMS (~0.24 ms) and drawing (0.3 to 0.7 ms) together contribute about 3 to 3.5 ms and are not worth optimizing before the stages above.

6. NPU/AIE Activity and Power

6.1 AIE core activity

XRT AIE profiling captured all 304 tiles using the CORE_ACTIVE and CORE_GROUP_STALL event counters.

Metric	Value
Tiles measured	304
Average core activity	52.48%
Minimum / Maximum	52.21% / 52.77%

The tight spread across tiles indicates evenly distributed work. This value is measured AIE core activity, not a statement that only 52% of the NPU is in use.

Profiling overhead. Enabling AIE profiling inflated a 1920x1080 run from 7.12 ms (NPU) / 74.99 ms (frame) / 13.33 FPS to ~10 ms / ~136 ms / ~7.3 FPS, an 81% increase in frame latency. Utilization profiling and performance benchmarking must therefore be run separately; the XRT configuration was restored to the clean setting afterwards.

6.2 Power

Board rails were read through the INA226-based monitoring interface (VCCINT, VCC_SOC, VCC_PMC, VCC_RAM, VCC_PSFP, VCCAUX, MGTAVCC and others, 20 rails in total). A representative reading of the monitored-rail total was **3.378 W**. This is a single snapshot and should not be treated as average active power at any given resolution. A valid power-versus-resolution comparison requires continuous sampling during each test, averaged over the active inference period; energy per frame can then be derived.

7. Recommendations

Priority	Target	Rationale	Possible approach
1	JPEG read/decode	44.31 ms at 1080p (59% of frame)	Hardware or SIMD-accelerated decoder (e.g. libjpeg-turbo), decode-time downscaling (DCT scaling), multi-threaded decode, or camera/stream input in a lower resolution or raw format
2	Input preparation	Fixed ~12.2 ms, 1.7x the NPU stage	Optimize float-to-INT8 NHWC4 conversion with NEON/fixed-point code; fuse with letterbox; write directly into the NPU input buffer
3	Letterbox/resize	Up to 7.97 ms at 1080p	Combine resize with preparation in one pass; use optimized resize routines
4	Pipeline overlap	NPU idle most of the frame	Run read/decode, preparation, NPU and post-processing in separate threads with double buffering so throughput is limited by the slowest stage rather than their sum
5	Power study	Only a single reading exists	Continuous INA226 sampling per resolution; report W and mJ/frame

Indicative impact of pipelining

If stages were fully overlapped, throughput would be bounded by the slowest stage rather than the frame total. Using the measured stage times, the bound would be ~22.6 FPS at 1920x1080 (read-limited, 44.31 ms) versus 13.33 FPS today, and ~32.8 FPS at 1280x720 (30.50 ms) versus 17.76 FPS. These are estimates only: they assume stages are independent and enough CPU cores are available, and CPU contention at 95% busy may reduce the gain. Parallel decode of several frames would raise the bound further.

8. Limitations and Notes

- The 175.4 ms and 73.75 ms figures come from the VART runner, while the stage breakdown comes from the custom profiling application; direct comparison is indicative only. The ~66 ms gap between the 73.75 ms runner latency and the 7.1 ms NPU stage suggests substantial runner/input-handling overhead.
- Read/decode at 320x240 (9.52 ms) is high relative to image size and above the 640x480 letterbox cost pattern; file-system or first-access effects may contribute and are worth checking.
- Power was not sampled per test; no power-per-resolution or energy-per-frame claims can be made from the available data.
- CPU busy percentages are aggregate figures; per-core utilization would show whether the pipeline is single-thread bound.
- AIE activity (52.48%) was measured on the latest profile only and under profiling overhead, so it describes activity levels, not benchmark performance.

9. Conclusion

Moving YOLOv7 from a CPU-involved path to an NPU-focused path cut runner latency from 175.4 ms to 73.75 ms, and stage-level profiling shows the VEK280 NPU completes a 640 x 640 inference in a stable ~7.1 ms. End-to-end performance, however, ranges from 27.01 FPS (320x240) to 13.33 FPS (1920x1080) because of CPU-side work: at 1080p, 44.31 ms reading the image and 12.22 ms preparing input, against 7.12 ms of NPU time. The next gains lie in the CPU input pipeline and in overlapping it with NPU execution, not in further NPU tuning.